

# Quality Assurance I

## Quality Assurance II

### Zero-Mock Testing Policy

All tests use real methods exclusively [ @martin2008clean; @meszaro s2007xunit]. No `unittest.mock`, no `MagicMock`, no patch decorators. Tests that require external services (Ollama, network) use `pytest.mark` markers for conditional execution. The philosophical motivation—*analogizing mock objects to Simmons et al.'s researcher degrees of freedom* [ @simmons2011falsepositive] and the pre-registration remedy [ @nosek2018preregistration]—is developed fully in the Zero-Mock Tradeoff discussion. To our knowledge, no prior research software engineering framework has formalized a zero-mock policy as an *architectural invariant enforced by pipeline gates*, where mock usage is not merely discouraged but structurally prevented from passing the build. The following example, drawn from the infrastructure test suite, illustrates zero-mock compliance:

```
def test_discover_infrastructure_modules_returns_nonempty(t):  
    # Real filesystem, real YAML parsing - no MagicMock and  
    modules = discover_infrastructure_modules(REPO_ROOT)  
    assert len(modules) >= 8  # actual subpackages
```